ChatGPT

TaskForge: Reliable Asynchronous Job Runner Prototype Plan

Executive Summary: TaskForge is a cloud-based job orchestration platform designed to execute background tasks reliably and scalably. It accepts long-running jobs (e.g. video processing, data analysis) via a REST API and processes them asynchronously with robust failure recovery and observability. Key design points are **at-least-once delivery**, **idempotent processing**, **dead-letter handling**, and optional **transactional outbox** for zero-loss semantics. Our architecture uses a database with an outbox table, a message queue (e.g. Redis Streams or RabbitMQ), a pool of worker processes, and a monitoring dashboard. We implement retries with exponential backoff plus jitter to avoid thundering-herd failures, and we treat any growing Dead-Letter Queue (DLQ) as an alert signal of underlying issues【5†L108-L116】【17†L141-L146】. Advanced features include AI-driven failure diagnosis and dynamic scaling. The prototype will be demonstrated by submitting jobs, forcing failures (killing workers or injecting errors), and showing automatic recovery (retries, DLQ routing) and observability of metrics (queue depth, latency, failure counts).

Problem Definition

Modern applications (video sites, SaaS platforms, e-commerce, IoT) often need to perform **long-running tasks** (e.g. transcoding, report generation, bulk emails) without blocking the user's request. A **job queue** decouples job submission from execution: the producer **enqueues** tasks and immediately returns, while one or more worker processes **consume** and execute tasks later【6†L72-L80】. However, naive queues suffer hard problems when things go wrong: workers can crash mid-task, external APIs may timeout, duplicate deliveries can happen, or a message may never be acknowledged. In practice, "*every distributed system runs on at-least-once delivery*"【5†L71-L75】, so any job might run twice. We must therefore build the system to **handle failures gracefully**: ensuring no tasks are lost, avoiding duplicate side-effects, and providing clear diagnostics when jobs fail repeatedly【5†L71-L75】【8†L180-L189】. TaskForge's goal is to provide a **resilient background-processing service** that guarantees eventual job completion (or clear dead-letter reporting) under realistic failure modes.

Target Users & Use Cases

TaskForge is aimed at **developers and platform teams** building scalable services that need reliable background jobs. Typical users include:

- **SaaS and PaaS providers:** Companies offering APIs or web apps that perform processing (e.g. AI inference, file processing, notifications) will use TaskForge to decouple and scale jobs.
- **E-commerce sites:** Handling payment webhooks, order fulfillment, email notifications without blocking checkout.
- **Media and content platforms:** Transcoding videos, generating thumbnails, indexing content, all asynchronously.
- **Data pipelines:** Running long ETL tasks, batch reports, or analytics jobs.
- **Enterprise integrations:** Syncing data to CRM, calling third-party APIs with retry logic.

Common scenarios (citing [6†L79-L83][8†L319-L328]) include: user uploads a video (worker must transcode, thumbnail, etc.), sending transactional emails, processing payment webhooks, generating large reports offline, and syncing with external systems with fault tolerance. In all cases, TaskForge ensures that these background tasks are retried and recovered automatically, and that administrators can monitor progress and failures.

Reliability Goals and Guarantees

- **Delivery semantics (At-least-once):** We assume at-least-once delivery (the queue will retry until acknowledged)【5†L71-L75】【6†L119-L127】. Hence the system's goal is to *safely handle duplicate deliveries*, not attempt impossible exactly-once guarantees.
- **Idempotency:** Every job's handler must be idempotent: its side effects (database changes, external calls) are safe to repeat【5†L71-L75】【19†L94-L100】. We will enforce this via unique *deduplication keys* (e.g. storing a unique job ID in a database or cache)【17†L136-L139】. For example, if a "send email" job is retried, the email code checks if that email was already sent and skips if so.
- **Transactional Outbox (atomic enqueue):** To prevent the "dual-write trap" (API writes to DB then crashes before queueing the job), we'll use a **Transactional Outbox pattern**【17†L55-L63】【19†L61-L69】. The API handler will insert the job details into an `outbox` table within the same database transaction as any related business data. A separate dispatcher process (or CDC) will tail the outbox and enqueue jobs into the message broker, guaranteeing that no committed job is ever silently lost【17†L55-L63】【19†L61-L69】.
- **Retries and Backoff:** Jobs will be retried on failure up to a configurable limit. Retries use *exponential backoff* plus *full jitter* to avoid synchronized retry storms【17†L141-L146】【5†L108-L116】. For example, with initial delay 1s: retries happen after ~1s, 2s, 4s, 8s, 16s, each time adding random jitter.
- **Dead-Letter Queue (DLQ):** After exceeding the max retry count, tasks are routed to a Dead-Letter Queue rather than retrying endlessly【8†L180-L189】. The DLQ holds failed jobs (with error info) for manual inspection. Importantly, we monitor the DLQ: a growing DLQ depth is treated as an operational alert, signalling that something is systematically failing【5†L112-L116】【34†L725-L732】. The DLQ is an "inbox someone has to review," not a dump【8†L187-L190】.
- **Visibility timeouts / Leases:** For brokers like SQS or Redis Streams, tasks are temporarily hidden when a worker fetches them. We must set the *visibility timeout* longer than the worst-case job time (ideally the P99 execution time)【25†L25-L33】【25†L37-L44】. A too-short timeout leads to duplicate processing (another worker picks up the same job prematurely)【25†L25-L33】. We will measure job durations and configure this margin accordingly.
- **Failure Notification:** We will propagate job status updates (including failures) to clients or admins. For critical failures, the system will log errors and (optionally) notify on-call via email/Slack. We plan at least to alert if the DLQ depth exceeds a threshold.
- **Observability:** We will collect metrics (job counts by status, latencies) and logs. In particular, we track P95/P99 job latencies, queue depth vs. message age, and DLQ size【34†L725-L732】. These indicators (e.g. "AgeOfFirstAttempt" metric in AWS SQS) reveal hidden backlogs【34†L718-L726】. We set SLOs on tail-latencies and error rates.

In summary, TaskForge guarantees that **all enqueued jobs will eventually succeed or move to DLQ with explanation**, and that duplicate executions produce no inconsistent state【5†L71-L75】【34†L748-L752】. These guarantees are satisfied by at-least-once delivery, idempotent handlers, outbox atomicity, retries with jitter, and dead-letter handling.

Failure Modes

We identify key failure scenarios and our mitigation:

- **Worker crash mid-task (no ack):** If a worker dies or is killed, the broker will re-enqueue the job after its visibility timeout. The job might run twice. *Mitigation:* Rely on idempotent handlers so a second run has no additional effect【5†L71-L75】【19†L94-L100】.
- **Worker crash after task complete but before ack:** The job did succeed, but the ack was lost. Broker will retry, but idempotency will ensure no double side-effect【5†L71-L75】【19†L94-L100】.
- **Slow jobs (timeout):** If a job takes longer than the visibility timeout, another worker may fetch it. *Mitigation:* Set timeout to safely exceed the worst-case (e.g. 2× p99)【25†L25-L33】【25†L37-L44】. Implement a lease-renew or heartbeat if needed.
- **Corrupt or invalid payload ("poison pill"):** A message that always fails (due to bad data) would occupy a worker indefinitely. *Mitigation:* Limit retries (e.g. 5 attempts); on final failure, route to DLQ for manual analysis【8†L180-L189】. Alert on DLQ.
- **Dependency outage (e.g. DB down, third-party API down):** If external services fail, retries with backoff and jitter will spread out attempts【17†L141-L146】. If the outage persists beyond max attempts, the job moves to DLQ with error context.
- **Queue broker failure:** We will use a durable queue (e.g. Redis with AOF persistence, or RabbitMQ with mirrored queues) so that restarts do not lose messages. On broker outage, the system may temporarily stop processing; jobs remain persisted and will resume when the broker is back.
- **Database crash:** Using transactions and outbox, a database outage could delay new jobs. On DB recovery, unprocessed outbox entries will be dispatched. Clients may experience a brief inability to enqueue jobs, which we can handle via retries at the API layer.
- **Network partition:** If components lose connectivity, messages may pile up in the queue. Because we rely on at-least-once delivery, the system will catch up once the network returns. If a dispatcher or worker can't reach the queue, it will try again (or the client will retry enqueue).
- **Data loss in dual writes:** To avoid this, we use the transactional outbox so that no job is lost between DB and queue【17†L55-L63】.

These modes define our testing: we will *intentionally induce* worker crashes, slow tasks, and exceptions to verify recovery flows. Throughout, we assume "design for duplicate" – i.e. we accept that tasks may be delivered twice and focus on preventing inconsistent state【34†L782-L789】.

Architecture

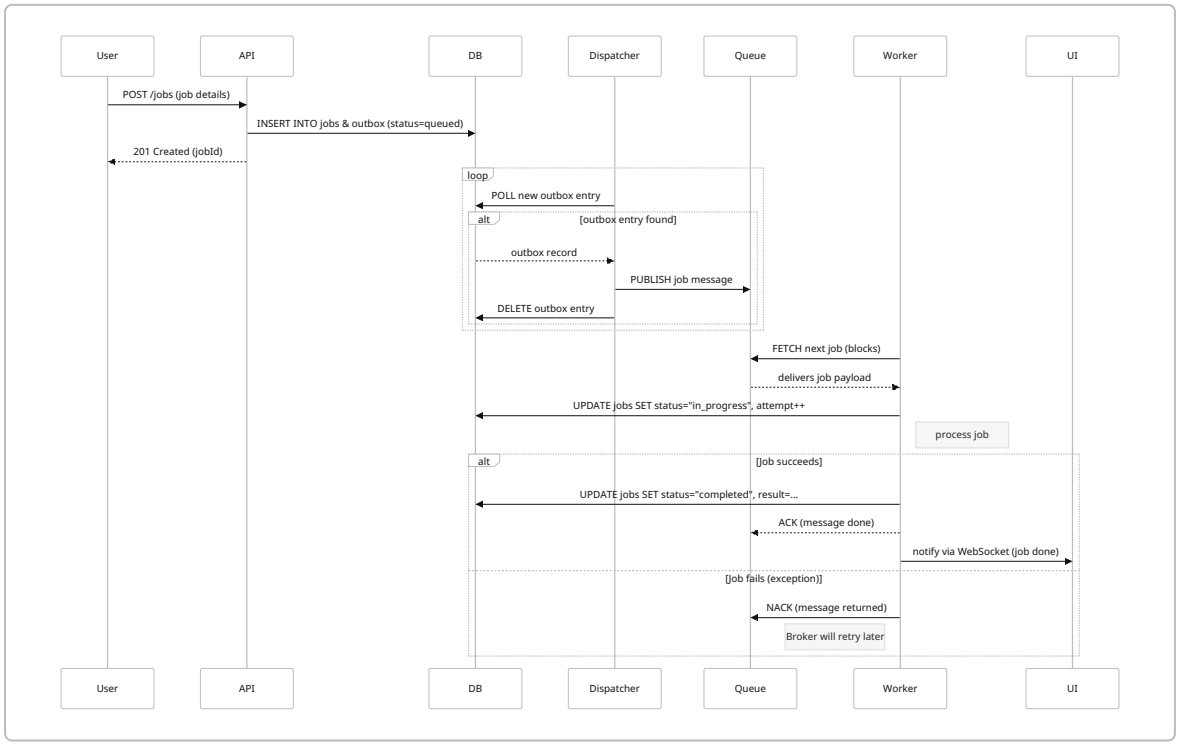
Overview: TaskForge consists of:

- **Frontend Dashboard:** A React+TypeScript web app (deployed e.g. via Vite) to visualize job queues, run metrics, and controls. It connects to the backend via REST APIs and WebSockets.
- **API Service:** A backend (e.g. FastAPI/Python or Spring Boot/Java) that provides endpoints to submit jobs, query status, cancel jobs, and manage tenants. It writes jobs to a **PostgreSQL** database (including the outbox table) and immediately returns a Job ID.
- **Database:** A relational DB (Postgres) storing `jobs` and `outbox` tables. The `jobs` table records each job's metadata (ID, type, payload, status, attempts, last error, etc.). The `outbox` table holds new jobs to be dispatched atomically.
- **Dispatcher:** A service (could be a loop or Debezium CDC process) that polls the `outbox` table for new entries and publishes them to the **Message Queue**. It marks outbox entries as sent or

deletes them after confirming enqueue. This ensures an atomic handoff from DB to broker【17†L55-L63】【19†L61-L69】.

- **Message Queue:** A broker (e.g. Redis Streams, RabbitMQ, or Kafka). We favor Redis Streams for simplicity (via Redis lists or XADD/XREAD), or RabbitMQ for its built-in DLX support. The queue holds pending jobs until a worker pulls them.
- **Worker Pool:** One or more stateless worker processes (e.g. Python/Celery workers or Go/Asynq tasks, or Node/BullMQ workers). Each worker listens on the queue, takes a job, updates the `jobs` table to *in_progress*, processes the task, then updates `jobs` to *completed/failed*. On failure it either lets the broker retry (by not ack-ing or explicitly NACK), up to the retry limit. Workers also enforce idempotency (e.g. checking a Redis or DB dedupe key)【17†L136-L139】.
- **Recovery Mechanisms:**
- **Heartbeats:** Workers can periodically refresh a lease or heartbeat timestamp for long jobs, preventing premature redelivery.
- **Dead-Letter Handler:** When a job hits its max attempts, the worker/service sets its status to *failed* and optionally copies it to a DLQ view for manual handling.
- **Monitoring & Metrics:** We integrate **Prometheus** for metrics (job counts, latencies, worker health) and **Grafana** for dashboards. The frontend dashboard also shows real-time stats (using WebSockets or polling) for queue depth and job states【8†L271-L280】.
- **Optional AI Assistant:** A component (triggered by the UI) that takes a failed job's logs and invokes an LLM (OpenAI/Anthropic) to summarize the error and suggest fixes. It runs offline (no automatic execution of suggestions) and filters output for accuracy.

Below is a **high-level sequence diagram** for job submission and processing:



In this architecture, **data flow** is: User → API → (DB+Outbox) → Dispatcher → Queue → Worker → DB → UI. We also include **sequence charts** in slides to illustrate the reliability mechanisms (heartbeats, retries, DLQ moves).

Feature List

MVP Features (Critical, must-implement):

- **Job Enqueue/Dequeue Pipeline:** Endpoints to submit jobs and workers that process them. Basic status tracking.
- **Retry with Backoff:** Configurable retry count; exponential backoff with jitter【5†L108-L116】.
- **Idempotency Keys:** Prevent duplicate side effects by deduplication (e.g. Redis NX or unique DB constraint)【17†L136-L139】.
- **Dead-Letter Queue:** After max retries, move jobs to a DLQ for review【8†L180-L189】.
- **Dashboard & Metrics:** UI showing queue depth (pending, active, failed counts), job statuses, processing times【8†L277-L280】.
- **API Endpoints:**
- `POST /jobs` – submit a job (returns job ID).
- `GET /jobs/{id}` – retrieve job status/result.
- `GET /jobs?status=failed` – list failed jobs.
- (Optional) `DELETE /jobs/{id}` – cancel a queued job.
- **WebSockets for Live Updates:** Push job completion/failure events to the dashboard in real time.
- **Persistent Storage:** PostgreSQL tables for jobs/outbox; Redis/RabbitMQ persistence.
- **Observability:** Collect metrics (via Prometheus) and logs (structured logging).
- **Basic Authentication:** Secure the API (e.g. JWT or API keys).

Advanced Features (If time allows):

- **Priority Scheduling:** Support job priorities or deadlines; ensure urgent jobs jump ahead (RabbitMQ priority queues or sorted Redis).
- **Autoscaling Workers:** Spawn more worker processes/containers when queue backlog grows (could integrate with Kubernetes HPA or a simple script) to handle spikes.
- **AI Failure Assistant:** Integrate an LLM to explain failures in plain language. For example, given a failed job ID, output: "The job failed due to a timeout while calling the email API. Consider increasing the timeout or retrying later." This uses actual error logs to avoid hallucination.
- **Predictive Alerts:** Analyze job queue trends; e.g. forecast when queue depth will hit some threshold or when an SLA might be missed.
- **Multi-tenancy:** Support multiple organizations by partitioning data (e.g. tenant_id column, Row-Level Security) and isolating resources.
- **SLA Enforcements:** Allow setting an SLA (e.g. 95% of jobs < 5 min); reject or reroute jobs if latency too high.
- **Audit Trail:** Immutable log of all job state changes for compliance.

Feature Backlog (prioritised):

Priority	Feature	Description / Benefit
P1 (MVP)	Basic job pipeline	Enqueue, dequeue, update status (see above). Foundational to all other features.
P1 (MVP)	Retry + DLQ	Automatic retries with exponential backoff; dead-letter after N attempts【8†L180-L189】.
P1 (MVP)	Idempotency Keys	Deduplication to make retries safe【17†L136-L139】.

Priority	Feature	Description / Benefit
P1 (MVP)	Monitoring Dashboard	Real-time view of queue depth and job statuses (pending/ active/failed).
P1 (MVP)	REST API (Submit/ Status)	Interfaces for clients to enqueue jobs and poll status.
P2 (Enh.)	Priority Queues	Allow urgent tasks to preempt others (e.g. via RabbitMQ priority).
P2 (Enh.)	Autoscaling Workers	Increase/decrease worker count based on backlog size or CPU usage.
P2 (Enh.)	AI Failure Analysis	LLM that reads logs of a failed job and suggests causes/ solutions.
P3 (Enh.)	Analytics & Forecast	Usage charts, predicted queue growth, SLA breach alerts.
P3 (Enh.)	Multi-tenant Isolation	Logical separation of job data per user/org, with access control.
P3 (Enh.)	Rate Limiting	Prevent abuse by limiting job submissions per client.

These priorities ensure we first deliver a **working, robust core** (P1) before layering on extra polish. Each feature's complexity is gauged to fit the project timeline.

Technology Stack Options (trade-offs)

Layer	Options (Examples)	Pros & Cons
Frontend	*React+TypeScript* (with Vite)	(+) Fast development, rich ecosystem, type safety. (−) Some boilerplate; requires bundling. React is popular and works well with real-time UIs.
	Vue.js	(+) Simpler learning curve, reactive UI. (−) Smaller community than React.
Backend	*Python* (FastAPI + Celery/Asynq)	(+) Quick prototyping, many libraries. Celery (with Redis/ RabbitMQ) is battle-tested for jobs. (−) Celery has known pitfalls on retries; requires careful config. Asynq (Go) is faster but new.
	Java (Spring Boot + RabbitMQ/Kafka)	(+) Strong typing, mature ecosystem (Spring Batch/Boot, Kafka libraries). (−) More verbose; longer development time. Good for large teams.
	Node.js (Express/Nest + BullMQ)	(+) Unified JS stack (frontend + backend), BullMQ on Redis is easy and supports retries/DLQ. (−) Async JavaScript complexities; not as performant for CPU-heavy tasks.

Layer	Options (Examples)	Pros & Cons
Queue Broker	*Redis Streams* (with BullMQ or custom)	(+) Very easy to set up (Docker), low-latency, supports persistence via AOF. Good for moderate scale. (−) As an in-memory store, needs RAM; streams slower than Kafka under massive load.
	RabbitMQ	(+) Mature AMQP broker, built-in support for TTL, dead-letter exchanges, priorities. Stable under high load. (−) More components to manage; needs tuning of durability vs speed.
	Apache Kafka	(+) Excellent throughput and fault-tolerance (logs on disk, consumer offsets). (−) Complex to set up, overkill for simple jobs; lacks easy message requeue (requires manual offset control).
	AWS SQS/SNS (cloud)	(+) Fully-managed, auto-scaled, built-in at-least-once (and FIFO for exactly-once within batch). (−) Vendor lock-in; offline demo impossible; visibility timeouts needed.
Database	*PostgreSQL*	(+) ACID transactions (needed for outbox), JSONB support, mature, widely available. (−) Set up a cluster for HA is non-trivial in hackathon timeframe.
	MySQL or MariaDB	Similar pros/cons to Postgres (ACID, widely used).
	MongoDB	(+) Flexible schema (store jobs as documents). (−) We lose ACID multi-write guarantees, making outbox harder.
Worker Language	*Python* (Celery, RQ, Dramatiq)	(+) Rich ecosystem, easy gluing of libraries (AWS SDK, etc.). (−) Python GIL limits concurrency; Celery reliability issues if misconfigured.
	Go (Asynq, Machinary)	(+) High performance, built-in concurrency. Asynq is a solid library with Redis. (−) Smaller community, fewer tutorials than Celery.
	Java (Spring @Async, JobRunr)	(+) Enterprise reliability; libraries like JobRunr for Java exist. (−) More boilerplate; not as fast to iterate.
Real-time Updates	*WebSockets* (Socket.io, ws)	(+) Low-latency push to clients for status changes. (−) Additional server complexity; must handle connection scaling.
	Server-Sent Events (SSE)	(+) Simpler than WebSockets for one-way updates. (−) Not bi-directional; fine for push notifications from server to UI.
Monitoring	*Prometheus + Grafana*	(+) Industry standard; rich metrics support, alerting. (−) Requires setup of Prometheus server and exporters.
	Custom Dashboard (Recharts, Chart.js)	(+) Fully integrated UI experience. (−) Reinventing wheel for graphs; lacks easy alerting.

Layer	Options (Examples)	Pros & Cons
Deployment	*Docker Compose*	(+) Simple setup for prototype (one command bring-up). (−) Not auto-scaled; manual container management.
	Kubernetes	(+) Production-like scaling (HPA, multi-node). (−) High operational overhead; likely overkill for hackathon.
	Serverless / Cloud Run	(+) No server management; easy horizontal scale. (−) Cold-starts, per-request billing. Hard to demo offline.
AI Integration	*OpenAI GPT-4 (API)*	(+) Strong language understanding out of box. (−) Internet dependency, possible hallucination (we'll constrain answers).
	Local LLM (e.g. Llama2)	(+) Fully offline; no per-call cost. (−) Weaker model; large memory requirement.

Stack Trade-off Highlights: We will likely choose **React+TS** for the frontend (fast dev, great community), **FastAPI (Python)** or **Node.js** for the API (quick prototype, async-friendly), **Redis Streams** (via BullMQ) as the job broker (simple Docker setup) and **PostgreSQL** for persistence and outbox. This combo is easy to develop on a laptop/cloud VM and has good libraries. We cite best practices: *"pick the tool by delivery semantics, not by popularity"*【5†L118-L123】. For example, Celery+Redis or BullMQ both give at-least-once semantics, while something like Temporal would give stronger orchestration guarantees but require much more learning and complexity. For time-boxed work, Redis/BullMQ or RabbitMQ are safer choices.

Data Models and APIs

Database Tables:

- **jobs** table (Postgres) with columns:
- `id (UUID/serial)`: primary key.
- `type (text)`: job type/name (e.g. "send_email", "transcode_video").
- `payload (JSONB)`: job-specific data (e.g. email address, video URL).
- `status (enum)`: {queued, in_progress, completed, failed, cancelled}.
- `attempts (int)`: number of attempts so far.
- `max_attempts (int)`: configured retry limit (default e.g. 5).
- `priority (int)`: (optional) higher = urgent.
- `created_at, updated_at (timestamps)`.
- `last_error (text)`: error message from last failure attempt.
- `result (JSONB)`: output or metadata from successful run.

- `tenant_id (UUID)`: if multi-tenant support is needed.

- **outbox** table (Postgres):

- Similar columns as `jobs`, but only for new jobs not yet dispatched.
- E.g. `job_id (FK to jobs)`, `payload`, `dispatched (bool)`, `dispatched_at`.
- The API writes to `outbox` in the same transaction as `jobs`.

- Dispatcher will delete or mark `dispatched = true` after publishing to queue.

- **dead_letter** table (optional):

- Could be a view on `jobs where status='failed'`.

- Or a separate table mirroring `jobs` schema to archive irrecoverable jobs.

- **worker_heartbeats** table (optional):

- If we implement explicit heartbeating: (`worker_id`, `last_heartbeat_at`).
- The dispatcher/monitor can detect dead workers.

REST API Endpoints (examples):

- `POST /api/jobs`
  Request: JSON `{ "type": "string", "payload": {...}, "priority": number }`
  Response: 201 Created with `{ "job_id": "uuid" }`.
  Action: Inserts a new row in `jobs` (status=queued) and in `outbox`.

- `GET /api/jobs/{id}`
  Response: Job details (type, status, attempts, result or last_error).

- `GET /api/jobs?status=failed`
  List all failed jobs (for admins).

- `DELETE /api/jobs/{id}`
  Cancel a job: If it's still queued, mark as `cancelled` to prevent processing.

- **Auth:** All endpoints require an API key or token identifying the caller (and tenant). The API verifies the caller's rights to read/write that job.

WebSocket API:

- `WS /ws/jobs`
  Upon connecting (with auth), the client can subscribe to job updates. We push messages like `{"job_id": "...", "status": "...", "percent_complete": ..., "error": "..."}` whenever a job changes state. This enables live dashboard updates.

(All REST APIs and WS channels are documented in the prototype's README or API reference.)

Testing & Validation Plan

We will rigorously test fault tolerance through **simulated chaos** and monitoring of key metrics:

- **Unit Tests:** For business logic and handlers (ensuring idempotency logic works, retry logic triggers, etc.).
- **Integration Tests:** End-to-end tests where we enqueue jobs and assert final states. For example, submit a "succeed" and a "fail" job, ensure the first completes, the second ends in DLQ after N retries.
- **Fault-Injection Tests:** Automate scenarios:

- Kill a worker process mid-job and assert the job eventually completes by another worker (no loss).
- Slow down or drop network to the DB/queue and verify recovery.
- Corrupt the job payload and ensure it moves to DLQ after retries (no crash of system).
- Exceed visibility timeout by delaying worker ack, ensure no duplicate beyond designed window.
- **Metrics Verification:** Use Prometheus to gather:
- `job_queue_depth` (pending jobs count) vs. `job_age`.
- `jobs_processed_total`, `jobs_failed_total`.
- Histograms of job processing time (p95/p99 latency).
- `dlq_depth` over time. We will set alerts (even manually in the demo) if DLQ grows【34†L725-L732】.
- **Chaos Testing:** Possibly use a tool like **Chaos Monkey** or custom scripts to randomly kill services or pause them.
- **Load Testing:** If feasible, simulate concurrent job submissions (e.g. 1000 jobs) and measure throughput. Check how queue length grows and how quickly workers drain it.
- **Data Integrity Checks:** After recovery scenarios, verify idempotency: e.g. if sending emails, confirm each address got exactly 1 email despite retries.

Validation Criteria: We aim to prove that under test conditions, *no job is lost or silently duplicated*. We will log every job ID and ensure each appears exactly once in the database as completed or failed. We will also verify observability: e.g. intentionally cause 5 jobs to fail and confirm the system raises no false alerts (jobs simply move to DLQ with logged errors).

Citations: We follow recommended practices: calculate visibility timeouts from job p99【25†L37-L44】, treat DLQ growth as alert【34†L725-L732】, and assume every job may run twice【34†L782-L789】. Observability metrics follow industry advice on tracking queue age and tail latencies【34†L706-L714】【34†L725-L732】.

Security & Multi-Tenancy

- **Authentication/Authorization:** All API calls (and WS) require a token identifying the user/team. We attach a `tenant_id` to each job. The backend checks that the requester can only manage their own jobs. In the DB, we enforce RLS (row-level security) on the `jobs` and `outbox` tables to isolate tenants.
- **Data Protection:** All network traffic (API, WS, DB connections) uses TLS. The job payload and results may contain sensitive data, so we encrypt the database at rest (common in Postgres setups).
- **Input Validation:** We validate and sanitize all job payloads on receipt. For example, if jobs involve user data or URLs, we check for malicious content. We do not allow arbitrary code in job definitions (jobs are identified by predefined types, not user-provided scripts).
- **Least Privilege:** Workers run under a service account with only needed DB permissions. They cannot access other tables. The monitoring stack has limited access as well.
- **Audit Logging:** Each state change (enqueue, start, complete, fail) is logged with timestamps and user IDs. This audit log can be reviewed to detect suspicious patterns.

Multi-tenancy considerations:

- Each tenant sees only their own jobs in the UI. The front-end will be parameterised by tenant context.
- If on Kubernetes, we could even use separate namespaces per tenant for worker pods (optional extra isolation).

Scalability and Cost Estimates

Scalability: TaskForge is horizontally scalable:

- **Workers:** We can run multiple worker processes across machines or containers. The queue distributes jobs (Redis Streams or RabbitMQ with multiple consumers can parallelize work). We will design the worker to be **idempotent and stateless**, so adding more workers immediately increases throughput.
- **Queue Broker:** Redis can be scaled by increasing memory, or using Redis Cluster. RabbitMQ can be clustered. For very high workloads, Kafka (partitioning) could be considered, but is likely beyond hackathon scope.
- **Database:** For prototype, a single Postgres instance is fine. In production, one would use a managed cluster or read replicas.

Cost (cloud estimate): We plan an open-source stack that can run on a modest VM. If using AWS:

- **Compute:** Suppose 2 m5.large instances (~$0.096/hour each) for one API server and one DB. Over a month (~720h), ~$138 per instance.
- **Queue:** If using Redis on an EC2, ~ $50/mo. If AWS SQS, it's ~$0.40 per million messages 【24†L82-L85】.
- **Storage:** Postgres storage ~$0.10/GB-month.
- **Monitoring:** Prometheus + Grafana on the same VM.

Given these, a minimal operational cost could be <$200/mo for moderate load. In a hackathon setting, we likely use a single machine or free-tier cloud VM, keeping costs near $0.

We will note for judges: managed services ease scaling (e.g. AWS SQS + Lambda can auto-scale), but require internet and incur costs. For offline demo, we stick to local containers.

Constraints and Risks

- **Time:** Building a full production-quality system is vast. We must focus on the core reliability features. We assume a **multi-week sprint** pre-hackathon to code the prototype (with ~3 developers). If our timeline shrinks (e.g. 1–2 weeks), we will cut advanced features (AI, autoscale) and polish only the core pipeline.
- **Complexity:** Outbox+CDC adds complexity. As a risk, we could simplify: for MVP, write to queue synchronously in the API (accepting tiny risk of dual-write), while noting the outbox as a future improvement【17†L55-L63】.
- **Offline Demonstration:** If judges inspect the demo without internet, our AI assistant cannot call external APIs. We'll either disable it in offline mode or use a local LLM with downloaded model. We must clearly demarcate which components require connectivity.
- **Team Expertise:** If the team has no prior experience with certain tech (e.g. Kubernetes, AI models), we should swap to easier choices (e.g. Docker Compose, skip LLM).
- **Data Loss Risk:** Misconfiguring retries (too few) or timeouts (too short) can cause job loss or duplication. We mitigate this by following the documented patterns【25†L25-L33】【17†L141-L146】 and testing thoroughly.
- **Third-Party Failure:** If using any external API (e.g. email service in a job), that service's downtime would cascade. We will mock such dependencies in demo or use a stable test endpoint.
- **Security Risk:** Unvalidated input could lead to injection (SQL/NoSQL/command). We will sanitize inputs and use parameterized queries.

In short, the biggest risk is **over-scoping**. We will mitigate by defining a clear Minimum Viable Product and stopping there before adding bells-and-whistles.

Implementation Roadmap

We envision a **2-4 week timeline** to build the prototype, with overlapping tasks:

- **Week 1 – MVP Core:**
- Set up project repos, Docker Compose (Postgres, Redis/RabbitMQ, app containers).
- Define DB schema (`jobs` and `outbox` tables).
- Implement API endpoints for job submission and status query.
- Implement a simple worker that reads from queue and processes a dummy task.

- Basic UI to submit a job and display its status (without metrics).
  Milestone: End-of-week, we can enqueue a job and see it complete in DB.

- **Week 2 – Reliability Features:**

- Add **retries**: configure broker retries or implement in worker (e.g. Celery retries, or requeue logic).
- Implement **exponential backoff + jitter** on failures【17†L141-L146】.
- Enforce **idempotency**: e.g. use Redis `SETNX` or DB unique key to skip duplicates【17†L136-L139】.
- Implement **Dead-Letter handling**: after N retries, flag job status = failed and save it.

- Extend UI: show jobs in queue, running, failed.
  Milestone: System can handle a failing job: after retries it appears in the “Failed” list.

- **Week 3 – Observability & UI:**

- Integrate Prometheus client in the app to emit metrics (job counts, latencies).
- Create Grafana dashboard or UI charts (Recharts) for queue depth and histograms.
- Add WebSocket updates: frontend listens for status changes to update UI in real time【8†L271-L280】.
- Polish API (error handling, authentication).

- Begin **testing**: write unit/integration tests for retry and idempotency logic.
  Milestone: Live dashboard shows a simulation of jobs flowing, metrics trending.

- **Week 4 – Advanced / Polish:**

- **AI Assistant:** Hook up an LLM (e.g. OpenAI GPT) to analyze a selected failed job’s logs. (Have prompts that instruct the model to only use provided info.)
- **Priority & Scaling:** If time permits, implement job priorities and a basic worker auto-scaler (could be as simple as running extra worker processes when queue length > X).
- **End-to-End Tests:** Simulate chaos (kill worker, DB restart) and ensure recovery.
- Fix bugs, add comments, write documentation.
  Milestone: Prototype is demo-ready with all promised features working on the deployed system.

Throughout, we use **Agile milestones** (reviews every few days) and keep the hackathon judges’ criteria in mind: innovation, technical implementation, and demo quality.

Demo Script (Offline Presentation)

1. **Introduction (Dashboard View):** Show the TaskForge dashboard. Explain its purpose: “This is TaskForge, a reliable background job runner.”
2. **Submit Jobs:** Use the UI to submit a batch of jobs (e.g. “send email” tasks). The dashboard shows them moving from *Queued* → *In Progress* → *Completed*. Emphasize non-blocking UI (user freed immediately).
3. **High Load Scenario:** Submit many jobs at once (or simulate via a script). Show queue depth spike and new workers automatically starting (if auto-scaling implemented) or multiple workers picking jobs. Point out metrics like jobs/sec and latency charts updating.
4. **Worker Failure Test:** In the admin console (or shell), kill one worker process. The dashboard shows one worker “Down”. Then watch: one or more *In Progress* jobs fail to ack; after timeout, they re-enter the queue and another worker picks them up. End result: all jobs still complete. (Demonstrates resilience.)
5. **Poison Job:** Submit a job designed to fail (e.g. wrong payload). Observe: it retries 5 times (with increasing delays) and then moves to *Failed*. Dashboard highlights it in a DLQ view.
6. **Dead-Letter Monitoring:** Show an alert or notification triggered by the DLQ depth increase (or simply note that the operator sees the new failed job). Explain that this signals a human should investigate.
7. **AI Failure Analysis (if enabled):** Click on the failed job and invoke the “Explain Failure” button. The UI shows an AI-generated explanation: e.g. “Error: External API timed out after 30s. Likely cause: third-party downtime. Suggestion: retry later or increase timeout.” This demonstrates practical AI usage (grounded in actual logs).
8. **Idempotency Proof:** To prove idempotency, simulate duplicate delivery: forcibly resend a completed job’s message. Show that the worker checks a dedupe key and does nothing (or logs “skipped”), ensuring no duplicate side-effects.
9. **Security / Multi-tenant (optional):** Log in as a different user; show that previous jobs are not visible (or attempt to cancel another user’s job is forbidden).
10. **Metrics Summary:** Quickly flash the Grafana/dashboard metrics panels: queue backlog chart, tail-latency graph, DLQ gauge. Note that the DLQ count is low (good) and job durations meet the defined SLO.
11. **Code Walkthrough (if time):** Briefly show critical snippets (e.g. the retry config, the idempotency check in code) to highlight robust engineering.

This demo emphasizes *failure scenarios and recovery* (steps 4–7) rather than just happy paths. Judges will see evidence of system behavior (jobs re-queued, metrics updating) rather than static slides.

Success Criteria

We define clear success metrics for the prototype:

- **Functional Correctness:** All jobs submitted should finish in either *completed* or *failed* (DLQ) state. No job should silently vanish. In tests, every submitted Job ID appears exactly once in the final results.
- **Reliability Metrics:** Under simulated load/failures, measure:
- *Completion Rate:* ≥99% of non-corrupt jobs should eventually complete.
- *Duplicate Side-Effects:* Confirm 0 unintended duplicates (e.g. via logs or test assertions).
- **Recovery Behavior:** When a worker is killed, the job it was processing must complete with another worker (within a short time). The “recovered” job count should equal the killed job count.

- **Dead-Letter Accuracy:** Jobs with permanent errors (test fault) should *only* appear in DLQ after exhausting retries. No good job should end up in DLQ. (We verify by checking business conditions.)
- **Performance (Prototype Scale):** While not aiming for massive throughput, we test that ~100 concurrent jobs can be handled and drained by adding more workers. Queue backlog should clear under added capacity.
- **Observability Indicators:** Metrics (e.g. tail latency, queue age) must reflect the actual state. For example, after queuing 10 jobs, the queue depth metric should show ~10 until workers pick them up. A spike in failures should be visible in the metrics.
- **Demonstration Points:** In the demo, success means all scripted interactions work as planned. The system should *noticeably* log or display each event (job accepted, started, retried, failed), and our manual triggers (killing a worker) produce the correct automatic response (no human intervention needed).
- **AI Explanation Quality:** (If implemented) The assistant's advice should be coherent and grounded. We judge qualitatively (e.g. factual correctness in describing the error and sensible suggestion).

Finally, we treat the hackathon scoring criteria: **innovation and depth.** Building a fully working pipeline with proven recovery behavior and monitoring should impress by demonstrating mastery of distributed-systems principles. We aim to be able to answer "Yes – this job can be safely re-run and the system handles it" confidently, showing we followed the *one rule*: design as if every job might run twice 【34†L782-L789】.

Sources: We based this design on contemporary engineering best practices for job queues【5†L71-L79】【17†L155-L164】, tutorials on retries and DLQs【8†L180-L189】, and patterns like the Transactional Outbox【17†L55-L63】【19†L61-L69】. The above plan cites guidance from industry experts and recent 2026 references to ensure technical accuracy.
